

Reviewer Pack — Minimal Rank of Monodromy Representations vs Boolean Circuit...

Entry bb4250a93b9e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 22:48:26 UTC

Minimal Rank of Monodromy Representations vs Boolean Circuit Size for XOR-AND Networks

Entry ID: bb4250a93b9e **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-22 22:48:26 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Monodromy Theory) **Field B** (complexity object): Complexity Theory: Boolean Circuit Complexity for XOR-AND Networks

Statement:

[‘For every Boolean circuit C computing an XOR-AND function with n variables, there exists a monodromy representation M of degree d such that the minimal rank of the associated vector bundle is upper bounded by a function $f(d) = O(n^2 \log n)$.’, ‘The minimal rank of M is lower bounded by $\Omega(n^2 \log n)$ ’, ‘For any instance with n variables, there exists a Boolean circuit C computing an XOR-AND function that requires at least d gates.’]

Rationale (proposer’s reasoning):

[‘Monodromy representations provide a rich algebraic structure that can be used to encode the complexity of computational problems. The connection between monodromy and circuit size could potentially reveal new insights into the structure of Boolean functions.’, ‘The minimal rank of a monodromy representation is related to the geometry of the base variety, which may offer a new perspective on the complexity of XOR-AND networks.’, ‘This conjecture bridges algebraic geometry with computational complexity, exploring the potential of using geometric invariants to understand circuit complexity.’]

Taxonomy category: ACC_SIPSER (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 163fdf535c4e0150

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all n variables, the mean minimal rank of monodromy representations across 30 seeds is less than or equal to $O(n^2 \log n)$ and there are no instances where the minimal rank exceeds this bound.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	UNCERTAIN	0.90	HITS	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "monodromy representation" AND "Boolean circuit complexity" - "minimal rank" "vector bundle" "algebraic geometry" AND "circuit size" - "XOR-AND networks" AND "lower bound" monodromy

Top relevant hits considered: - [http://arxiv.org/abs/1603.07248v3] Flat vector bundles and open coverings - [http://arxiv.org/abs/1501.02540v1] Equivariant vector bundles on complete symmetric varieties of minimal rank - [http://arxiv.org/abs/1010.1825v2] Equations for secant varieties via vector bundles - [http://arxiv.org/abs/2509.16077v1] On the Number of Control Nodes of Threshold and XOR Boolean Networks - [s2:2504.05807] Low-Complexity AoI-Optimal Status Update Control with Partial Battery State Information in Energy Harvesting IoT Network - [s2:10.4208/csiam-am.so-2022-0005] Non-Lipschitz Attack: A More Sparse Adversarial Attack via Non-Lipshitz ℓ_p Regularization

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return abs(a*b) // gcd(a, b)

def matrix_multiply(A, B):
    m, k = len(A), len(B[0])
    n = len(B)
    C = [[0] * k for _ in range(m)]
    for i in range(m):
        for j in range(k):
            for l in range(n):
                C[i][j] += A[i][l] * B[l][j]
    return C
```

```

def matrix_add(A, B):
    m = len(A)
    n = len(A[0])
    C = [[A[i][j] + B[i][j] for j in range(n)] for i in range(m)]
    return C

def matrix_subtract(A, B):
    m = len(A)
    n = len(A[0])
    C = [[A[i][j] - B[i][j] for j in range(n)] for i in range(m)]
    return C

def matrix_transpose(A):
    m = len(A)
    n = len(A[0])
    C = [[A[j][i] for j in range(m)] for i in range(n)]
    return C

def determinant(A):
    if len(A) == 1 and len(A[0]) == 1:
        return A[0][0]
    det = 0
    sign = 1
    for i in range(len(A)):
        submatrix = [row[:i] + row[i+1:] for row in A[1:]]
        det += sign * A[0][i] * determinant(submatrix)
        sign *= -1
    return det

def inverse(A):
    det = determinant(A)
    if det == 0:
        raise ValueError("Matrix is singular")
    m, n = len(A), len(A[0])
    adjoint = [[0] * n for _ in range(m)]
    for i in range(m):
        for j in range(n):
            submatrix = [row[:j] + row[j+1:] for row in A[:i] + A[i+1:]]
            cofactor = determinant(submatrix)
            adjoint[j][i] = cofactor * (-1) ** (i + j)
    return matrix_multiply(adjoint, Fraction(1, det))

def gaussian_elimination(A):
    m, n = len(A), len(A[0])
    for i in range(m):
        max_row = i
        for j in range(i+1, m):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]
        pivot = A[i][i]
        for j in range(n):
            A[i][j] /= pivot
        for j in range(m):
            if j != i:

```

```

        factor = A[j][i]
        for k in range(n):
            A[j][k] -= factor * A[i][k]
    return A

def rank(A):
    rref = gaussian_elimination(A)
    m, n = len(rref), len(rref[0])
    rank = 0
    for i in range(m):
        if any(rref[i][j] != 0 for j in range(n)):
            rank += 1
    return rank

def xor_and_circuit(n):
    if n == 1:
        return [[0], [1]]
    left = xor_and_circuit(n // 2)
    right = xor_and_circuit(n - n // 2)
    circuit = []
    for a in left:
        for b in right:
            circuit.append([a[0] ^ b[0]])
            circuit.append([a[0] & b[0]])
            circuit.append([a[0] ^ b[1]])
            circuit.append([a[0] & b[1]])
    return circuit

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 5 + (seed % 4) * 5
    if n < 5 or n > 40:
        return {
            "metric_name": "Minimal Rank of Monodromy Representations",
            "metric_value": None,
            "instances_tested": 0,
            "conjecture_holds": False,
            "counterexample": "n_out_of_range"
        }
    circuit = xor_and_circuit(n)
    rank_value = rank(circuit)
    return {
        "metric_name": "Minimal Rank of Monodromy Representations",
        "metric_value": rank_value,
        "instances_tested": 1,
        "conjecture_holds": rank_value <= n**2 * math.log(n),
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(1, 1000000) for _ in range(30)]
    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")

```

```

results.append(trial_result)

mean_value = sum(result["metric_value"] for result in results if result["metric_value"] is not None)
std_value = math.sqrt(sum((result["metric_value"] - mean_value)**2 for result in results if result["metric_value"] is not None) / len(results))
support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

if all(result["conjecture_holds"] for result in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif any(not result["conjecture_holds"] for result in results):
    first_failing_seed = next((seed for seed, result in zip(seeds, results) if not result["conjecture_holds"]), None)
    print(f"RESULT: FALSIFIED counterexample=\"n_out_of_range\" first_failing_seed={first_failing_seed}")
else:
    print(f"RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_214e9e28.py", line 145, in <module>
    trial_result = run_trial(seed)
                    ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_214e9e28.py", line 130, in run_trial
    circuit = xor_and_circuit(n)
               ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_214e9e28.py", line 108, in xor_and_circuit
    left = xor_and_circuit(n // 2)
            ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_214e9e28.py", line 108, in xor_and_circuit
    left = xor_and_circuit(n // 2)
            ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_214e9e28.py", line 108, in xor_and_circuit
    left = xor_and_circuit(n // 2)
            ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_214e9e28.py", line 115, in xor_and_circuit
    circuit.append([a[0] ^ b[1]])
                    ~~~~
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing any data, which means that the conjecture could neither be supported nor falsified based on the available information. | next: Re-run the test with proper error handling to ensure it completes without crashing and produces results for analysis.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15008
2	propose	ollama_remote	glm4:latest	0	0	14646
3	preregistration	ollama_remote	glm4:latest	0	0	9138
4	novelty	ollama_remote	glm4:latest	0	0	9116
5	novelty	ollama_remote	glm4:latest	0	0	9581
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13495
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10688
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7951
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16175
10	judge	ollama_remote	glm4:latest	0	0	11600

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 117398 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/bb4250a93b9e.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/bb4250a93b9e.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/bb4250a93b9e.tar.gz (if generated)